

Distributed Random Forest

Progetto dei corsi Sistemi Distribuiti e Cloud Computing & Machine Learning

Giada Pica

Macroarea di Ingegneria Informatica Università degli Studi di
Roma Tor Vergata
Veroli, Italy
giada.pica@alumni.uniroma2.eu

Simone Manisi

Macroarea di Ingegneria Informatica Università degli Studi di
Roma Tor Vergata
Grottaferrata, Italy
simone.manisi@alumni.uniroma2.eu

Abstract—Questo documento presenta l'architettura, le scelte progettuali e l'implementazione di un sistema di Random Forest distribuito basato su architettura Master-Worker, realizzato in ambiente Amazon AWS. L'obiettivo del progetto è distribuire le fasi computazionalmente onerose di training e inferenza di un modello di Machine Learning su più nodi di calcolo, valutandone prestazioni, scalabilità e robustezza operativa. Il sistema è sviluppato in Python e utilizza gRPC per la comunicazione tra componenti, Docker per la containerizzazione dei servizi e Amazon EFS come storage condiviso per dataset, split, alberi addestrati, metriche, metadata e manifest dei modelli. L'architettura prevede un cluster di master, nel quale un solo nodo leader coordina le operazioni critiche, e un insieme di worker responsabili dell'addestramento distribuito degli alberi e della produzione di predizioni parziali. Il progetto integra, inoltre, meccanismi di fault tolerance, tra cui leader election, heartbeat dei worker, persistenza degli artifact e recupero dei job incompleti, al fine di ridurre la perdita di progresso in caso di crash di worker o master leader.

Introduzione

Negli ultimi anni, la crescente disponibilità di dati e la diffusione di applicazioni basate su Machine Learning hanno reso sempre più importante la progettazione di sistemi capaci di eseguire elaborazioni su larga scala in modo efficiente, scalabile e affidabile. In numerosi contesti applicativi moderni, come l'analisi sanitaria i modelli di apprendimento automatico devono essere addestrati e interrogati su quantità di dati sempre maggiori. In questi scenari, l'esecuzione su una singola macchina può rappresentare un limite significativo, sia in termini di tempo di calcolo sia in termini di capacità di gestione delle risorse.

MODELLO RANDOM FOREST

Un modello Random Forest è un algoritmo di machine learning supervisionato che utilizza alberi decisionali per migliorare accuratezza, robustezza e capacità di generalizzazione. Viene impiegato sia per task di classificazione che di regressione.

È formato da un insieme di alberi decisionali indipendenti, ognuno addestrato su un sottoinsieme casuale dei dati forniti. Ogni albero fa una propria previsione; la Random Forest combina tutte le previsioni per ottenere il risultato finale, a seconda del task l'aggregazione avviene in maniera diversa.

Per la classificazione: si usa la maggioranza dei voti.

Per la regressione: si calcola la media delle predizioni.

Il modello usa la tecnica del bootstrap aggregating (bagging), ovvero:

- Campiona in modo casuale con ripetizione una parte dei dati per costruire ogni albero.

- Seleziona in modo casuale anche un sottoinsieme delle variabili a ogni divisione ("feature randomness").

L'uso del caso riduce la correlazione tra gli alberi, rendendo il

modello molto stabile, poco sensibile al rumore e meno soggetto a overfitting rispetto a un singolo albero decisionale.

Per questo possiamo affermare che random forest funziona bene anche con dati rumorosi, con molte feature e con relazioni non lineari.

Vantaggi principali:

- Alta accuratezza
- Robustezza al rumore
- Resistenza all'overfitting
- Funziona bene senza troppa preparazione dei dati.

ARCHITETTURA MASTER-WORKER

Il master rappresenta il punto logico di controllo del sistema: gestisce l'avanzamento del processo, mantiene una visione complessiva dello stato dell'esecuzione e coordina le attività da svolgere. I worker costituiscono invece il livello operativo dell'architettura: ricevono unità di lavoro, le elaborano in modo autonomo e restituiscono i risultati prodotti. Questa separazione permette di ottenere una struttura ordinata e modulare, nella quale il comportamento globale del sistema non dipende da un'unica fase monolitica, ma dalla cooperazione tra più componenti specializzati. Il master organizza l'esecuzione; i worker, non devono conoscere l'intera logica del processo, ma solo il compito assegnato. L'adozione di questo modello rende il sistema più adatto a scenari in cui il carico computazionale può crescere nel tempo. Nuovi worker possono essere introdotti per aumentare la capacità di elaborazione, mentre il master conserva il ruolo di coordinamento e aggregazione. In questo modo, l'architettura fornisce una base generale per realizzare un sistema scalabile, distribuito e più flessibile rispetto a un'esecuzione centralizzata.

ARCHITETTURA DEL PROGETTO

Il sistema implementato adotta un'architettura distribuita di tipo Master-Worker, nella quale le responsabilità di coordinamento e quelle di calcolo sono separate in modo netto. Tale scelta consente di mantenere centralizzata la gestione dello stato globale del sistema, delegando invece ai nodi worker le operazioni computazionalmente più onerose, come l'addestramento degli alberi decisionali e l'esecuzione delle predizioni parziali durante la fase di inferenza.

L'architettura complessiva è composta da quattro elementi principali:

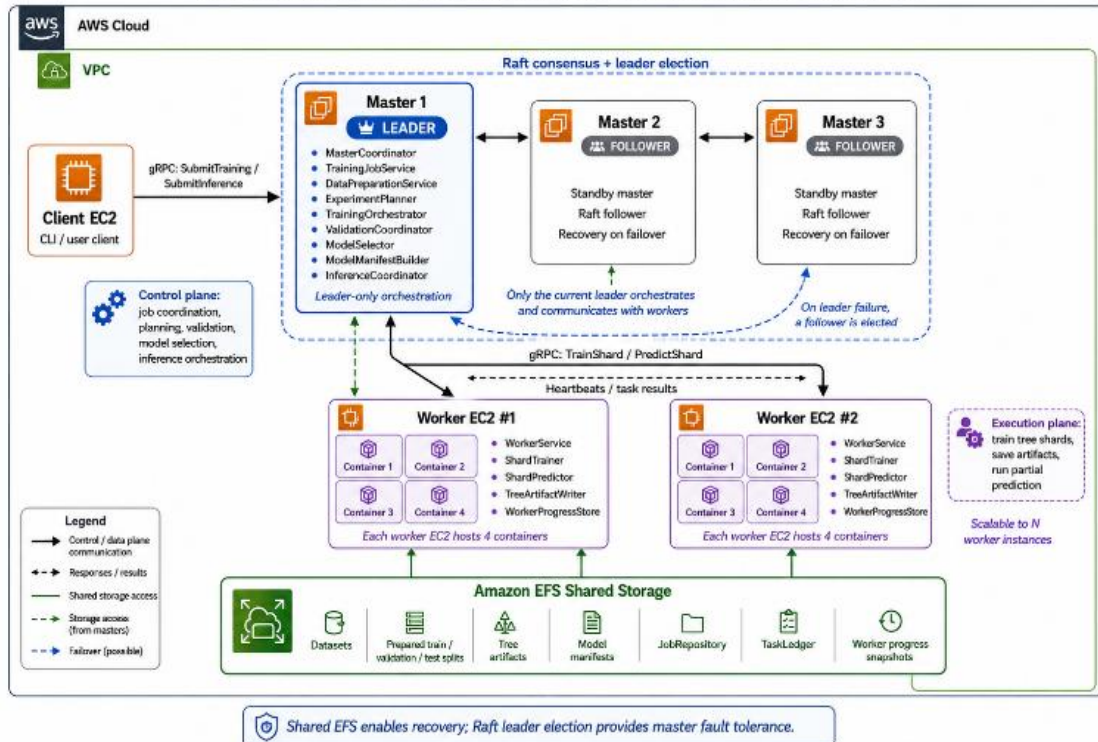
1. Il client
2. Il cluster di master
3. L'insieme dei worker
4. Lo storage condiviso

Il client rappresenta il punto di ingresso del sistema: esso invia al master:

- Le richieste di training o inferenza, specificando il dataset e il tipo di task di Machine Learning
- La variabile target
- I principali parametri necessari alla costruzione della foresta.

Distributed Random Forest System Architecture

Master/Worker architecture on AWS with Raft leader election and shared EFS storage



Il client non partecipa direttamente all'elaborazione distribuita, ma si limita a interagire con il sistema attraverso l'interfaccia esposta dal master. Per questo motivo, nell'infrastruttura realizzata, il client è stato eseguito su un'istanza EC2 di dimensioni ridotte, di tipo t3.micro, sufficiente a sostenere il ruolo di invio delle richieste e visualizzazione dei risultati.

Il cluster di master costituisce il piano di controllo dell'architettura. I master sono responsabili di:

- Ricevere le richieste
- Preparare il dataset
- Pianificare gli esperimenti
- Suddividere il lavoro in task
- Assegnare i task ai worker
- Raccogliere i risultati
- Pubblicazione del modello finale (*è possibile fare download lato client*)

All'interno del cluster, un solo nodo assume il ruolo di leader ed è autorizzato a coordinare le operazioni critiche. Gli altri nodi master rimangono disponibili per garantire continuità operativa in caso di guasto del leader, evitando che un singolo punto di fallimento comprometta l'intera esecuzione. Nel sistema realizzato sono stati predisposti tre nodi master, eseguiti su istanze Amazon EC2 di tipo m7i-flex.large, così da fornire risorse adeguate alla gestione del coordinamento e dei servizi applicativi.

I worker rappresentano il piano di esecuzione del sistema.

Nell'infrastruttura utilizzata sono stati previsti otto nodi worker, anch'essi basati su istanze EC2 di tipo m7i-flex.large, scelta coerente con il maggiore carico computazionale associato alle fasi di training e inferenza.

Nel sistema sviluppato viene utilizzato Amazon EFS come storage condiviso tra master e worker. Al suo interno vengono salvati i dataset preparati, gli split di training, validation e test, gli alberi addestrati, le metriche, i metadata e il manifest finale del modello.

Questa scelta permette ai diversi nodi di accedere a uno spazio comune, evitando di trasferire direttamente grandi quantità di dati nei messaggi gRPC e consentendo il recupero degli artifact già prodotti in caso di interruzioni.

Il client interagisce con il sistema inviando richieste di training o inferenza al master, che rappresenta il punto di ingresso e coordinamento dell'esecuzione. Il master leader comunica poi con i worker per distribuire le attività, monitorarne l'avanzamento e raccogliere i risultati prodotti. I worker, a loro volta, forniscono informazioni sul proprio stato, permettendo al master di rilevare eventuali nodi non più disponibili. Questa struttura rende possibile una gestione dinamica dei worker e costituisce la base per i meccanismi di fault tolerance.

Dal punto di vista infrastrutturale, il sistema è stato realizzato in ambiente cloud utilizzando Amazon Web Services. I nodi master, worker e client sono stati distribuiti su istanze Amazon EC2 nella regione europea di Stoccolma, in particolare nella zona di disponibilità eu-north-1b. I servizi applicativi sono stati eseguiti all'interno di container Docker, in modo da rendere più semplice il deployment dei componenti, garantire maggiore riproducibilità dell'ambiente di esecuzione e consentire l'avvio di più servizi sulla stessa istanza virtuale. Lo sviluppo del progetto è stato effettuato principalmente in Python, utilizzando PyCharm come ambiente di sviluppo.

Per la comunicazione remota sono stati utilizzati gRPC e Protocol Buffers. gRPC è stato impiegato per implementare le chiamate tra client, master e worker, mentre Protocol Buffers è stato utilizzato per definire in modo tipizzato i messaggi e i servizi esposti dai componenti distribuiti. Questa scelta consente di avere interfacce più chiare, messaggi strutturati e una comunicazione efficiente tra i nodi del sistema.

Le principali librerie Python utilizzate sono grpcio, grpcio-tools e protobuf per la comunicazione gRPC e la generazione degli stub a partire dai file .proto; numpy e pandas per la gestione dei dati; scikit-learn per la costruzione degli alberi decisionali, la generazione di dataset sintetici, le baseline locali e il calcolo delle metriche; joblib per la serializzazione degli alberi addestrati; pyarrow per la lettura e scrittura efficiente dei file Parquet; matplotlib per la produzione dei grafici sperimentali. Sono stati inoltre utilizzati pip e venv per la gestione dell'ambiente Python e delle dipendenze, insieme a script Bash per automatizzare build, avvio, aggiornamento e monitoraggio dei container master e worker.

DETTAGLI IMPLEMENTATIVI

L'implementazione del sistema è basata su una separazione netta tra il piano di controllo, gestito dal cluster di master, e il piano di esecuzione, rappresentato dai worker.

Il master non esegue direttamente le operazioni computazionalmente intensive sugli alberi, ma coordina il ciclo di vita dei job, prepara il dataset, pianifica gli esperimenti, suddivide la foresta in shard e assegna tali shard ai worker disponibili. Questa scelta permette di mantenere centralizzata la logica decisionale e, allo stesso tempo, di distribuire il carico computazionale tra più nodi.

DETTAGLI IMPLEMENTATIVI: MASTER

Il cluster master è composto da più istanze, tra le quali viene eletto un solo nodo leader. Il leader è l'unico nodo autorizzato a gestire le operazioni critiche del sistema. Gli altri master rimangono in esecuzione come nodi follower e non orchestrano direttamente job di training o inferenza. Questa impostazione evita condizioni di concorrenza e impedisce che più master assegnino simultaneamente gli stessi task o producano stati incoerenti. La fault tolerance del cluster master è realizzata tramite un meccanismo di leader election basato su Raft. Ogni nodo master partecipa al protocollo di elezione e mantiene informazioni sul proprio ruolo corrente, che può essere leader, follower o candidate. In caso di indisponibilità del leader, ad esempio a seguito di crash del container o dell'istanza, i nodi rimanenti avviano una nuova elezione e selezionano automaticamente un nuovo leader. Il nuovo leader può quindi riprendere il controllo del sistema senza richiedere un riavvio manuale dell'intera architettura. Il recupero dopo un crash del master è supportato dalla persistenza dello stato su storage condiviso.

Dal punto di vista implementativo, il ruolo del leader viene protetto da un componente di controllo, indicato come Leadership Guard, che impedisce l'esecuzione di operazioni riservate al leader da parte dei nodi follower. Prima di avviare attività critiche, il master verifica quindi il proprio stato all'interno del cluster. Se il nodo non è leader, la richiesta non viene eseguita localmente. Questo meccanismo consente di mantenere una semantica leader-only per training, validation, testing, inferenza e pubblicazione del modello finale.

Le informazioni fondamentali del job, gli artifact prodotti, gli alberi addestrati, i manifest dei modelli, le metriche e i metadata necessari alla ricostruzione dello stato non rimangono esclusivamente nella memoria volatile del master, ma vengono salvati in modo durevole. Quando un nuovo leader entra in funzione, esso può ispezionare lo stato persistito, individuare eventuali job incompleti e riprendere l'esecuzione a partire dagli artifact già disponibili. In questo modo, il sistema riduce la perdita di progresso computazionale e non è costretto a ripetere necessariamente l'intero training dall'inizio.

DETTAGLI IMPLEMENTATIVI: COMUNICAZIONE

La comunicazione tra master e worker è implementata tramite gRPC, utilizzando interfacce definite tramite Protocol Buffers. Questa scelta consente di definire contratti espliciti e tipizzati tra i componenti distribuiti, rendendo più chiara la struttura dei messaggi scambiati e semplificando l'evoluzione dell'architettura. Il master espone le funzionalità principali verso il client, come la sottomissione di job di training e inferenza, mentre i worker espongono operazioni dedicate all'esecuzione dei task assegnati. Durante la fase di training, il master prepara il dataset e persiste gli split su storage condiviso. Successivamente, il Training Orchestrator suddivide la Random Forest in shard, ciascuno dei

quali rappresenta un sottoinsieme deterministico di alberi da addestrare.

Ogni shard viene assegnato a un worker tramite una chiamata gRPC. Il messaggio non contiene l'intero stato globale del sistema, ma include le informazioni necessarie al worker per eseguire il proprio compito: identificativo del job, identificativo dell'esperimento, configurazione della foresta, intervallo o insieme di alberi assegnati, seed deterministici e URI degli artifact necessari sullo storage condiviso.

DETTAGLI IMPLEMENTATIVI: WORKER

Ogni worker riceve dal master un sottoinsieme del lavoro complessivo, corrispondente a un insieme di alberi della Random Forest da addestrare o da utilizzare in inferenza. Durante il training, i worker addestrano gli alberi assegnati e salvano gli artifact prodotti nello storage condiviso. Durante l'inferenza, invece, i worker caricano gli alberi di propria competenza, calcolano predizioni parziali e restituiscono al master i risultati da aggregare. Questa organizzazione consente di sfruttare il parallelismo naturale della Random Forest, distribuendo la costruzione e l'utilizzo degli alberi su più nodi. Il worker, ricevuto il task, carica i dati di training dallo storage condiviso, addestra gli alberi assegnati e salva ciascun albero come artifact persistente. Nel training della Random Forest, la generazione dei campioni bootstrap è demandata ai worker e non al master. Questa soluzione evita che il master debba generare e trasferire esplicitamente a ogni worker grandi matrici campionate, riducendo il traffico di rete e impedendo che il master diventi un collo di bottiglia computazionale. Al termine dell'esecuzione del train il worker, restituisce al master un risultato contenente l'esito dello shard, il numero di alberi completati, eventuali errori e i metadata degli artifact prodotti. Il master aggiorna quindi lo stato del job e del task ledger, verificando quali alberi risultano completati e quali, eventualmente, devono essere ripianificati.

La comunicazione gRPC viene utilizzata anche per la fase di inferenza. In questo caso, il master legge il manifest del modello finale, individua gli alberi che compongono la foresta selezionata e distribuisce ai worker il lavoro di predizione. Ciascun worker carica gli alberi di propria competenza, calcola predizioni parziali sui dati richiesti e restituisce i risultati al master. Il master aggrega infine le predizioni parziali, applicando la logica prevista dalla Random Forest: voto di maggioranza nel caso della classificazione e media dei valori predetti nel caso della regressione.

Un ruolo importante è svolto anche dai messaggi di heartbeat. I worker inviano periodicamente informazioni sul proprio stato, permettendo al master leader di mantenere un registro aggiornato dei nodi disponibili. Attraverso tali heartbeat, il sistema può rilevare worker non più raggiungibili, escluderli temporaneamente dalla pianificazione e riassegnare i task incompleti ad altri worker. Questo meccanismo contribuisce alla fault tolerance lato worker e permette al sistema di proseguire l'esecuzione anche in presenza di guasti parziali.

Nel complesso, l'implementazione combina tre elementi principali: leader election nel cluster master, comunicazione tipizzata tramite gRPC e persistenza degli artifact su storage condiviso.

La leader election garantisce continuità del piano di controllo in caso di crash del master leader; gRPC fornisce un'interfaccia chiara ed efficiente per la comunicazione tra nodi; lo storage condiviso permette di rendere durevoli dataset, alberi, metriche e manifest.

Fase	Contenuto gRPC	Contenuto su EFS
SubmitTraining	Richiesta logica, path dataset, iperparametri	Dataset sorgente, se collocato su EFS
Data preparation	Nessuna comunicazione verso worker	Split train/validation/test in parquet
Training shard	Range di alberi, seed, URI train, URI output	Training set e directory artefatti
Bootstrap	Flag e seed deterministici	Nessun dataset bootstrap persistito
Salvataggio alberi	Metadata di completamento	Alberi .joblib e metadata .json
Validation	URI validation, URI alberi, output dir	Feature validation, predizioni parziali
Test	URI test, URI alberi, output dir	Feature test, predizioni parziali
Inference	model_id, features_uri, URI alberi	Input inference, predizioni parziali e finali
Recovery	Nuovi shard per alberi mancanti	Ledger, job record, alberi completati

Sintesi della comunicazione per fase

ANALISI DEI DATASET E VALUTAZIONE DELLE PRESTAZIONI

ANALISI DEI DATASET

È stata predisposta un’analisi delle prestazioni predittive dei modelli addestrati considerando più dataset, più task di apprendimento e più scenari di preprocessing, con l’obiettivo di valutare il comportamento della Random Forest in condizioni sperimentali differenti. Il dataset *originale* utilizzato nel progetto è un dataset sanitario relativo al rischio di diabete. Esso contiene 100.000 istanze e 31 colonne, comprendenti:

- feature non categoriche*: variabili anagrafiche, comportamentali. Tra le feature sono presenti, ad esempio, genere, etnia, livello di istruzione, reddito, stato occupazionale...
- feature categoriche*: indice di massa corporea, pressione arteriosa, colesterolo, trigliceridi, glicemia, livello di insulina...

Il dataset contiene tre variabili particolarmente rilevanti per la valutazione sperimentale:

- diagnosed_diabetes**, utilizzata per la classificazione binaria,
- diabetes_stage**, utilizzata per la classificazione multiclasse,
- diabetes_risk_score**, utilizzata per la regressione.

La distribuzione della variabile **diagnosed_diabetes** presenta una prevalenza della classe positiva: nel dataset sono presenti circa 60.000 istanze positive e 40.000 istanze negative. La variabile **diabetes_stage**, invece, è fortemente sbilanciata: la maggior parte delle istanze appartiene alle classi “Type 2” e “Pre-Diabetes”, mentre classi come “Type 1” e “Gestational” sono molto meno rappresentate. Questa caratteristica rende il task multiclasse più complesso e impone di interpretare con attenzione metriche aggregate come accuracy e weighted F1-score, che possono risultare elevate anche quando il modello fatica sulle classi minoritarie.

SCENARI BASELINE E RISCHIO DI DATA LEAKAGE

Per rendere la valutazione più robusta, il progetto non utilizza un unico scenario sul dataset reale, ma introduce diversi scenari sperimentali:

Dataset scenarios overview

Scenario	Cosa fa	Colonne tolte	Righe finali	Feature originali rimaste	Feature dopo encoding
baseline_original	Dataset originale, nessuna rimozione	Nessuna	100.000	30	59
baseline_no_leakage	Rimuove le colonne considerate leakage rispetto alla diagnosi binaria	diabetes_stage, diabetes_risk_score	100.000	28	52
no_diagnostic_features	Rimuove leakage + marker diagnostici principali	diabetes_stage, diabetes_risk_score, hba1c, glucose_fasting, glucose_postprandial	100.000	25	49
no_diagnostic_extended	Versione ancora più restrittiva: rimuove anche insulin_level	diabetes_stage, diabetes_risk_score, hba1c, glucose_fasting, glucose_postprandial, insulin_level	100.000	24	48
clinical_only	Tiene solo feature cliniche generali	Tiene solo 13 feature cliniche, tutto il resto viene tolto	100.000	13	13
glucose_only	Tiene solo marker glicemici	Tiene solo hba1c, glucose_fasting, glucose_postprandial	100.000	3	3
diagnostic_noise_10pct	Rimuove leakage e aggiunge rumore ai marker diagnostici	Toglie diabetes_stage, diabetes_risk_score; altera hba1c, glucose_fasting, glucose_postprandial, insulin_level	100.000	28	52
diagnostic_noise_25pct	Come sopra, ma con rumore più forte	Toglie diabetes_stage, diabetes_risk_score; altera i marker diagnostici	100.000	28	52
diagnostic_noise_50pct	Come sopra, rumore ancora più forte	Toglie diabetes_stage, diabetes_risk_score; altera i marker diagnostici	100.000	28	52
imbalance_positive_80	Rimuove leakage e forza circa 80% classe positiva	Toglie diabetes_stage, diabetes_risk_score; sottocampiona la classe negativa	74.997	28	52
imbalance_positive_90	Rimuove leakage e forza circa 90% classe positiva	Toglie diabetes_stage, diabetes_risk_score; sottocampiona molto la classe negativa	66.664	28	52
imbalance_negative_80	Rimuove leakage e forza circa 80% classe negativa	Toglie diabetes_stage, diabetes_risk_score; sottocampiona la classe positiva	50.002	28	52

Lo scenario *baseline_original* mantiene tutte le colonne disponibili. Questo scenario è utile come punto di partenza, ma non è il più significativo dal punto di vista scientifico quando il target è **diagnosed_diabetes**, poiché include variabili come **diabetes_stage** e **diabetes_risk_score**, che sono direttamente collegate alla diagnosi e possono introdurre *data leakage*. Infatti, in questo scenario la classificazione binaria raggiunge una accuracy sul test set pari a 0.99970, valore eccessivamente elevato e non rappresentativo di una reale capacità predittiva generalizzabile.

Scenario	Task	Accuracy
Baseline_original	Classificazione	0.99970

Per questo motivo, lo scenario più rilevante per la classificazione binaria è *baseline_no_leakage*, nel quale vengono rimosse le colonne considerate troppo direttamente informative rispetto al target, in particolare **diabetes_stage** e **diabetes_risk_score**. In questo scenario il modello mantiene prestazioni elevate ma più realistiche: con 240 alberi, la baseline locale ottiene una test accuracy pari a 0.91515, un macro F1-score pari a 0.91370 e un weighted F1-score pari a 0.91594. La matrice di confusione mostra che il modello riconosce molto bene la classe negativa, ma presenta un numero non trascurabile di falsi negativi sulla classe positiva.

Scenario	Task	Accuracy	Macro F1	Weighted F1
Baseline no leakage	Classificazione	0.91515	0.91370	0.91594

Sono stati inoltre valutati scenari più restrittivi, come *clinical_only*, nel quale vengono mantenute soltanto alcune feature cliniche. In questo caso le prestazioni calano sensibilmente: la test accuracy è pari a 0.60725, mentre il macro F1-score scende a 0.46619. Tale risultato suggerisce che il sottoinsieme clinico selezionato non contiene informazioni sufficienti per ricostruire in modo efficace la diagnosi binaria. Inoltre, il valore di accuracy risulta molto vicino alla distribuzione delle classi nel dataset, pari approssimativamente al 60% per la classe positiva e al 40% per la classe negativa. Questo indica che il modello tende ad avvicinarsi al comportamento di un classificatore maggioritario, ottenendo una performance apparentemente discreta ma poco informativa dal punto di vista predittivo. Il basso macro F1-score conferma infatti che il modello non riesce a bilanciare correttamente il riconoscimento delle due classi e che il segnale predittivo più rilevante è contenuto anche in feature diagnostiche o comportamentali escluse da questo scenario.

Scenario	Task	Accuracy	Macro F1
Clinical_only	Classificazione	0.60725	0.46619

Nello scenario *baseline no leakage*, per il task di regressione con 240 alberi, la Random Forest ottiene sul test set un coefficiente R^2 pari a 0.72289, un MAE pari a 3.77876 e un RMSE pari a 4.78221. Questi valori indicano una buona capacità del modello di spiegare la variabilità del punteggio di rischio, pur lasciando una quota di errore residuo non trascurabile.

Il task di regressione sarà maggiormente trattato nella parte di tolleranza ai guasti.

Scenario	Task	R^2	RMSE	MAE
Baseline no leakage	Regressione	0.72289	4.78221	3.77876

ROBUSTEZZA DEL MODELLO

A partire dallo scenario no-leakage sono stati valutati ulteriori scenari di robustezza, progettati per osservare il comportamento della Random Forest in condizioni più difficili. In particolare, sono stati considerati esperimenti con: *noise injection*, forte *sbilanciamento delle classi* e task di *classificazione multiclasse*. Tali scenari permettono di analizzare non soltanto l'accuracy aggregata, ma anche metriche più informative come balanced accuracy, macro F1-score, weighted F1-score, precision e recall delle singole classi.

La *noise injection* consente di valutare la robustezza del modello rispetto a perturbazioni controllate dei dati.

Negli esperimenti distribuiti con 400 alberi, lo scenario con *noise al 10%* ottiene una validation accuracy pari a 0.89145, una balanced accuracy pari a 0.90341, un macro F1-score pari a 0.88984 e un weighted F1-score pari a 0.89250. Rispetto alla *baseline no-leakage* locale a 400 alberi, la perdita in accuracy è pari a circa 2.44 punti percentuali. La matrice di confusione mostra 7707 veri negativi, 294 falsi positivi, 1877 falsi negativi e 10122 veri positivi. Il modello mantiene quindi una buona capacità predittiva, ma l'aumento dei falsi negativi indica che il rumore riduce soprattutto la capacità di riconoscere correttamente una parte dei casi positivi.

Scenario	Accuracy	Balanced acc.	Macro F1	Weighted F1
Noise 10%	0.89145	0.90341	0.88984	0.89250

Con *noise al 25%*, la degradazione diventa più evidente. La validation accuracy scende a 0.86075, la balanced accuracy a 0.86646, il macro F1-score a 0.85777 e il weighted F1-score a 0.86189. Rispetto allo scenario *noise 10%*, l'accuracy diminuisce di circa 3.07 punti percentuali; rispetto alla *baseline no-leakage* a 400 alberi, la perdita complessiva è pari a circa 5.51 punti percentuali. La matrice di confusione contiene 7161 veri negativi, 840 falsi positivi, 1945 falsi negativi e 10054 veri positivi. L'aumento dei falsi positivi mostra che, con un livello di rumore più alto, il confine decisionale diventa meno stabile e il modello tende a commettere più errori in entrambe le direzioni.

Scenario	Accuracy	Balanced acc.	Macro F1	Weighted F1
Noise 25%	0.86075	0.86646	0.85777	0.86189

Il caso *noise 50%* nonostante il forte degrado, il modello in esecuzione con 400 alberi, ha dimostrato che non collassa completamente. È stata riscontrata un'accuracy pari a 0.82255 che indica che la Random Forest continua a catturare una parte rilevante del segnale predittivo. Ciò è coerente con la natura dell'algoritmo, la RF tende ad essere robusta a perturbazioni parziali delle feature. Seppure il dataset contiene diverse feature

correlate fra loro, ed il rumore rende perciò le feature diagnostiche meno affidabili, il modello può ancora sfruttare informazione residua contenuta nelle feature cliniche.

Scenario	Accuracy	Balanced acc.	Macro F1	Weighted F1
Noise 50%	0.82255	0.82132	0.85777	0.86189

SBILANCIAMENTO DELLE CLASSI

Un secondo gruppo di esperimenti riguarda lo *sbilanciamento della distribuzione delle classi*. Questo tipo di analisi è particolarmente importante perché, in presenza di classi fortemente sbilanciate, l'accuracy può diventare una metrica fuorviante. Un modello che predice quasi sempre la classe maggioritaria può infatti ottenere un'accuracy apparentemente elevata, pur avendo scarsa utilità sulle classi minoritarie.

Nello *scenario con 80% di istanze positive*, il modello distribuito con 400 alberi ottiene una validation accuracy pari a 0.88847, una balanced accuracy pari a 0.89129, un macro F1-score pari a 0.84488 e un weighted F1-score pari a 0.89422. La matrice di confusione contiene 2688 veri negativi, 312 falsi positivi, 1361 falsi negativi e 10639 veri positivi. Sebbene la classe positiva sia dominante, il modello mantiene una buona capacità di riconoscere anche la classe negativa, con recall pari a 0.89600 per la classe 0. Tuttavia, la precision della classe negativa è pari a 0.66387, indicando che una parte consistente delle predizioni negative è contaminata da esempi positivi classificati erroneamente. Il risultato è comunque superiore a quello di un classificatore banale che predicesse sempre la classe maggioritaria, il quale otterrebbe circa 0.80000 di accuracy.

Scenario	Accuracy	Balanced acc.	Macro F1	Weighted F1
80% Positivi	0.88847	0.89129	0.84488	0.89422

Lo *scenario con 90% di istanze positive* è invece molto più critico. L'accuracy è pari a 0.90047, apparentemente elevata, ma la balanced accuracy scende a 0.50325 e il macro F1-score a 0.48049. La matrice di confusione mostra 9 veri negativi, 1324 falsi positivi, 3 falsi negativi e 11997 veri positivi. Questo significa che il modello predice quasi sempre la classe positiva, riuscendo a riconoscere correttamente soltanto 9 esempi negativi su 1333. La recall della classe negativa è infatti pari a 0.00675. Il confronto con un classificatore banale che predicesse sempre la classe positiva è particolarmente significativo: tale classificatore otterrebbe circa 0.90002 di accuracy, quasi identica alla performance osservata. Questo esperimento dimostra chiaramente che l'accuracy, da sola, non è sufficiente per valutare modelli addestrati su dataset fortemente sbilanciati.

Scenario	Accuracy	Balanced acc.	Macro F1	Weighted F1
90% Positivi	0.90047	0.50325	0.48049	0.85419

Lo scenario con *90% di istanze negative* presenta invece un comportamento differente. Il modello ottiene una validation accuracy pari a 0.97180, una balanced accuracy pari a 0.92950, un macro F1-score pari a 0.95342 e un weighted F1-score pari a 0.97098. La matrice di confusione mostra 8001 veri negativi, 0 falsi positivi, 282 falsi negativi e 1718 veri positivi. In questo caso, nonostante lo sbilanciamento, il modello riesce a riconoscere bene anche la classe positiva minoritaria, con recall pari a 0.85900 e precision pari a 1.00000. Il risultato è molto superiore a quello di un classificatore banale che predicesse sempre la classe negativa, il quale otterrebbe circa 0.80002 di accuracy. Questo comportamento

suggerisce che, nel dataset utilizzato, i casi positivi possiedono pattern clinico-diagnostici più marcati e più facilmente separabili rispetto ai casi negativi, rendendo il modello più robusto quando la classe positiva è minoritaria rispetto al caso opposto.

Scenario	Accuracy	Balanced acc.	Macro F1
90% Negativi	0.97180	0.92950	0.95342

Per il task multiclasse è stato utilizzato lo scenario stage_multiclass_no_leakage, scegliendo **diabetes_stage** come variabile target. Rispetto alla classificazione binaria, questo esperimento introduce una difficoltà maggiore, poiché il modello deve distinguere tra più stadi della condizione diabetica e non soltanto tra pazienti positivi e negativi. Gli esperimenti mostrano che il problema multiclasse è più complesso rispetto alla classificazione binaria. Con una configurazione da 600 alberi, il modello raggiunge una test accuracy pari a 0.82960 e un weighted F1-score pari a 0.80552. Tuttavia, il macro F1-score è pari soltanto a 0.35659. Questa differenza indica che le metriche aggregate pesate sono fortemente influenzate dalle classi più frequenti, mentre il macro F1-score evidenzia le difficoltà del modello sulle classi minoritarie. Un comportamento analogo emerge anche dal modello distribuito con 400 alberi, che ottiene una validation accuracy pari a 0.85875 e un weighted F1-score pari a 0.85468, ma una balanced accuracy pari a 0.44767 e un macro F1-score pari a 0.45553. L'analisi per classe mostra infatti che le classi più numerose, come *“Pre-Diabetes”* e *“Type 2”*, vengono riconosciute con buone prestazioni, rispettivamente con F1-score pari a 0.82559 e 0.91881. Al contrario, le classi *“Gestational”* e *“Type 1”* ottengono F1-score pari a 0.00000, poiché il modello non riesce a riconoscerle correttamente.

Scenario	Accuracy	Macro F1	Weighted F1
Locale 600	0.82960	0.35659	0.80552
Distribuito 400	0.85875	0.45553	0.85468

Questi risultati confermano che anche in questo scenario l'accuracy, da sola, non è sufficiente per valutare il task multiclasse. Il modello raggiunge valori apparentemente elevati sulle metriche complessive, ma tali risultati sono condizionati dalla distribuzione sbilanciata delle classi. La Tabella riassume i principali risultati degli esperimenti di classificazione considerati.

Scenario	Esecuzione	Alberi	Accuracy	Balanced accuracy	Macro F1	Weighted F1	Osservazione principale
Baseline no-leakage	Locale	400	0.97580	0.92722	0.91439	0.91659	Scenario realistico di riferimento
Noise injection 10%	Distribuita	400	0.89145	0.90341	0.88984	0.89250	Degrado moderato, modello ancora stabile
Noise injection 25%	Distribuita	400	0.86075	0.86646	0.85777	0.86189	Degrado più marcato e aumento degli errori
Noise injection 50%	Distribuita	400	0.82295	0.82132	0.85777	0.86189	Degrado più marcato, ma segnale di robustezza al rumore
80% positivi	Distribuita	400	0.88847	0.89129	0.84488	0.89422	Sbilanciamento gestito in modo ancora accettabile
90% positivi	Distribuita	400	0.90047	0.50325	0.48049	0.85419	Accuracy fuorviante, quasi classificatore maggioritario
90% negativi	Distribuita	400	0.97180	0.92950	0.95342	0.97098	Buona separabilità della classe positiva minoritaria
Multiclasse diabetes_stage	Distribuita	400	0.85875	0.44767	0.45553	0.85468	Buone classi maggioritarie, classi rare non riconosciute

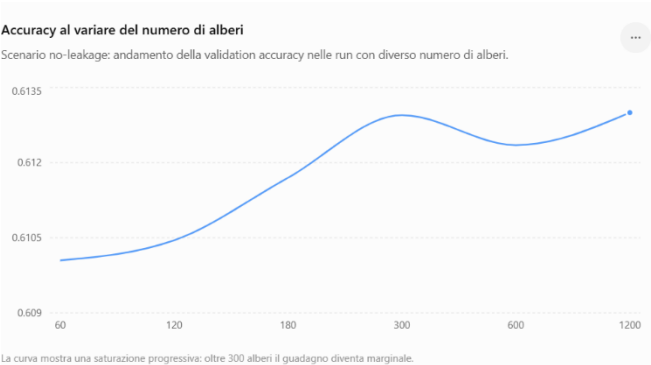
Riassunto metriche dataset di classificazione

Dal confronto emerge che la Random Forest distribuita mantiene una buona robustezza rispetto a livelli moderati di rumore, ma subisce un degrado progressivo quando il rumore aumenta dal 10% al 25%. La riduzione di accuracy e macro F1-score è coerente con l'aumento dell'incertezza nei dati e conferma che le metriche ML

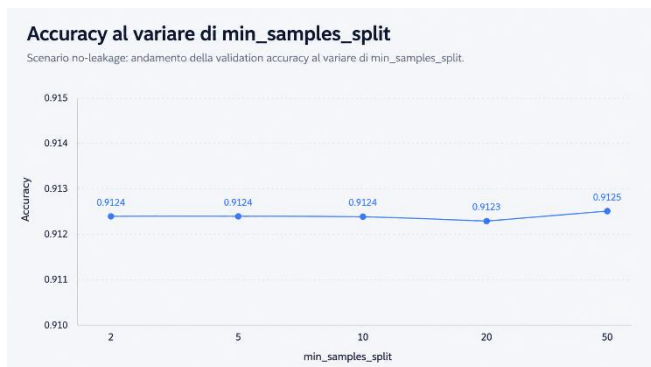
riescono a catturare l'impatto della qualità del dataset sul modello finale. Tuttavia, anche nello scenario noise 25%, il modello conserva una accuracy superiore a 0.86, indicando una certa stabilità dell'algoritmo rispetto a perturbazioni non estreme. Gli esperimenti sullo sbilanciamento mostrano invece un comportamento più articolato. Lo scenario 80% positivi mantiene prestazioni complessivamente buone e superiori al classificatore maggioritario. Lo scenario 90% positivi, al contrario, mostra il limite dell'accuracy come metrica principale: il valore di 0.90047 sembra elevato, ma la balanced accuracy di 0.50325 indica che il modello discrimina molto male la classe minoritaria. Lo scenario 90% negativi produce invece risultati molto migliori, suggerendo che la classe positiva, pur minoritaria, è caratterizzata da segnali diagnostici più riconoscibili. Un elemento trasversale interessante riguarda l'importanza delle feature. Nei diversi esperimenti distribuiti, le variabili più rilevanti risultano stabilmente **hba1c**, **glucose_postprandial** e **glucose_fasting**. Questo indica che il modello basa gran parte delle proprie decisioni, giustamente, su indicatori clinico-diagnostici fortemente associati al diabete. La degradazione osservata negli esperimenti di noise injection è quindi coerente con il fatto che alterare o rendere meno affidabili le feature diagnostiche riduce la separabilità tra le classi.

DESCRIZIONE IPERPARAMETRI USATI

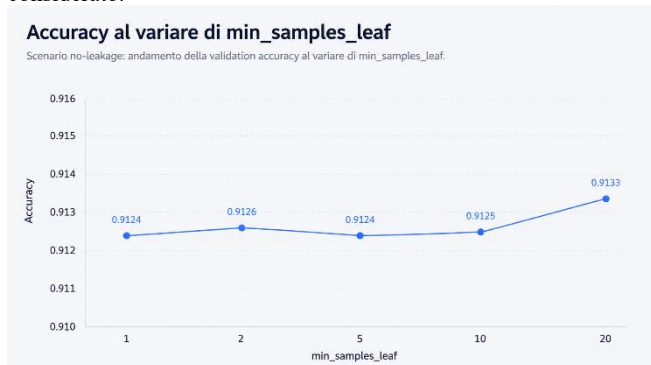
Il client consente di specificare a runtime la configurazione della Random Forest, permettendo di variare i principali **iperparametri** del modello. Per valutare l'impatto di tali scelte sulle prestazioni, sono stati eseguiti diversi esperimenti sullo scenario no-leakage, misurando la validation accuracy al variare di un parametro alla volta.



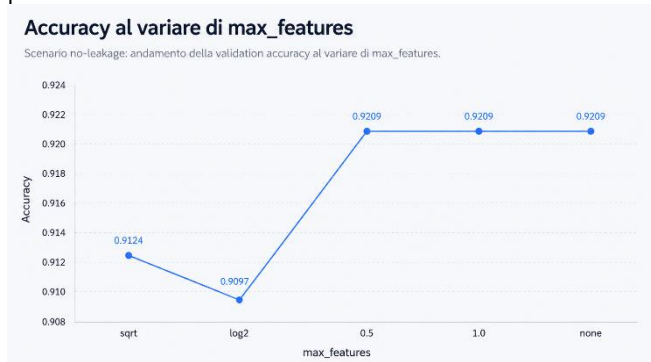
Una prima analisi ha riguardato il **numero di alberi della foresta**. Come mostrato nel grafico relativo all'aumento del numero di alberi, l'aumento del numero di stimatori produce un miglioramento progressivo ma contenuto della validation accuracy. Dopo una certa soglia, il guadagno diventa marginale rispetto al maggiore costo computazionale. Questo evidenzia il compromesso tra accuratezza e tempo di training, particolarmente rilevante in un sistema distribuito in cui ogni albero contribuisce al carico complessivo del job. Sono stati poi confrontati diversi criteri di split per la classificazione.



Il grafico relativo al numero minimo di samples per divisione mostra che la variazione di **min_samples_split** produce differenze molto limitate. I valori 2, 5 e 10 ottengono circa 0.9124 di accuracy, il valore 20 scende leggermente a 0.9123, mentre il valore 50 raggiunge circa 0.9125. Questo andamento suggerisce che il modello è poco sensibile a questo parametro nel range considerato.



Il grafico relativo al numero minimo di samples per foglia (**min_samples_leaf**) mostra un comportamento simile all'esperimento precedente. I valori più bassi mantengono accuracy intorno a 0.9124–0.9126, mentre il valore 20 raggiunge circa 0.9133. L'aumento del numero minimo di campioni per foglia può quindi introdurre una leggera regolarizzazione, riducendo la frammentazione dell'albero senza penalizzare la capacità predittiva.



Il grafico relativo all'iperparametro **max_features** mostra come la variazione di questo parametro abbia un impatto significativo sulle prestazioni. Infatti, le configurazioni sqrt e log2 ottengono rispettivamente circa 0.9124 e 0.9097, mentre i valori 0.5, 1.0 e none raggiungono circa 0.9209. Questo risultato indica che, per il dataset considerato, permettere agli alberi di valutare un numero maggiore di feature a ogni split migliora la capacità predittiva rispetto alle configurazioni più restrittive.



Il grafico relativo alla profondità massima mostra quanto questo parametro risulti uno degli iperparametri più influenti. Con **max_depth** pari a 3, l'accuracy è circa 0.9024; aumentando la profondità a 5 si raggiunge 0.9124, mentre con profondità 8, 10, 15 o senza limite esplicito si ottengono valori intorno a 0.9207–0.9209. Il miglioramento si concentra quindi nel passaggio da alberi poco profondi ad alberi mediamente più espressivi, mentre oltre profondità 8 il guadagno diventa quasi nullo. Nel complesso, gli esperimenti mostrano che non tutti gli iperparametri hanno lo stesso impatto sulle prestazioni. Parametri come **min_samples_split** e **min_samples_leaf** producono variazioni molto contenute, mentre **max_features** e **max_depth** influenzano in modo più evidente l'accuracy. In particolare, le configurazioni migliori osservate utilizzano una maggiore disponibilità di feature per split e alberi con profondità almeno pari a 8, oltre la quale il miglioramento tende a stabilizzarsi.

VALUTAZIONE DELLA SCALABILITÀ

Per valutare la scalabilità del sistema sono stati eseguiti più job di training variando il numero di worker disponibili e mantenendo costante la configurazione del modello. Le prove sono state effettuate su un dataset sintetico su task di *regressione* composto da 2.000.000 di campioni e 42 feature, addestrando una Random Forest da 400 alberi con profondità massima pari a 5. In questo modo, il carico computazionale rimane invariato tra le diverse esecuzioni e le differenze nei tempi possono essere attribuite principalmente alla diversa distribuzione del lavoro.



Nel grafico relativo al tempo di training in funzione al numero di worker viene riportato il tempo totale di training al variare della configurazione dei worker. La *notazione* utilizzata è del tipo NxM, dove N indica il numero di istanze EC2 e M il numero di container worker avviati su ciascuna istanza. *Ad esempio, la configurazione 4x2 corrisponde a 4 istanze con 2 container worker ciascuna, per un totale di otto worker.*

I risultati mostrano una riduzione significativa del tempo di training rispetto alla baseline 1x1, che richiede circa 49 m e 30 s. Il miglior risultato osservato è ottenuto con la configurazione 8x2, pari a 16 worker totali distribuiti su 8 istanze, che completa il training in circa 4 m e 19 s. Questo corrisponde a uno speedup di circa 11,5 volte rispetto all'esecuzione con un solo worker.

Il grafico evidenzia però che la scalabilità non dipende esclusivamente dal numero totale di worker, ma anche da come essi sono distribuiti sulle istanze. A parità di worker complessivi, configurazioni diverse possono produrre tempi molto differenti: ad

esempio, con 4 worker totali, la configurazione 4x1 risulta sensibilmente più veloce della configurazione 1x4. Questo suggerisce che concentrare più worker sulla stessa istanza può introdurre contesa sulle risorse locali, riducendo i benefici del parallelismo.

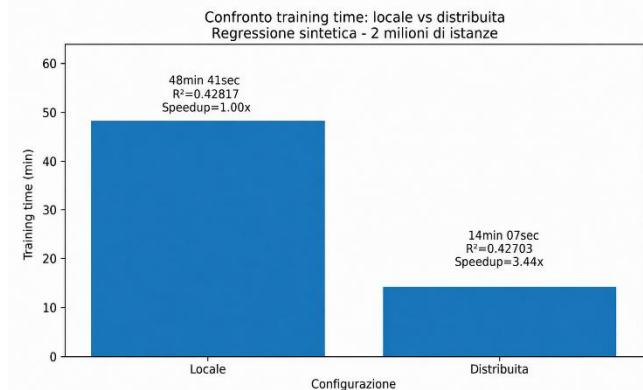
CONFRONTO LOCALE-DISTRIBUITO: TRAINING

Le esecuzioni in locale sono state completate su macchine con:

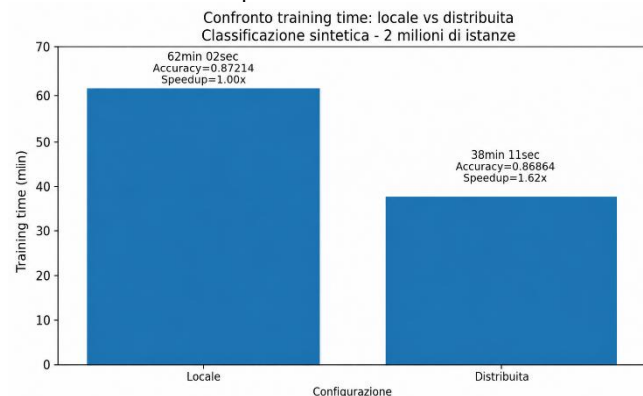
-I7 8th

-I5 5th

Queste informazioni hardware vanno a descrivere lo speedup come una metrica non simmetrica a livello hardware ma propriamente rispetto al tempo.

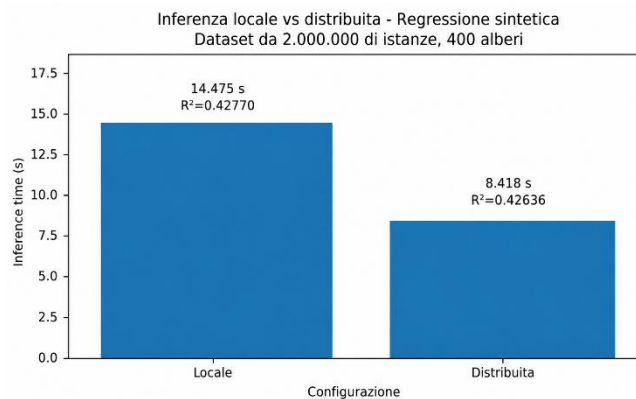


La regressione con dataset sintetico da 2.000.000 istanze rappresenta un carico computazionale adatto alla distribuzione: l'addestramento dei 400 alberi può essere suddiviso tra gli 8 worker, ciascuno dei quali contribuisce con 50 alberi completati, in una configurazione 2x4. L'assenza di retry e di alberi falliti conferma inoltre la stabilità dell'esecuzione distribuita. La differenza di R^2 è limitata e accettabile considerando che la Random Forest distribuita costruisce gli alberi in modo parallelo su nodi differenti. Nel report distribuito, il tempo totale end-to-end è pari a 14 m e 41 s, di cui 14 m e 4 s dedicati al training; ciò indica che la fase dominante rimane l'addestramento degli alberi, mentre le altre fasi hanno un impatto relativamente contenuto.



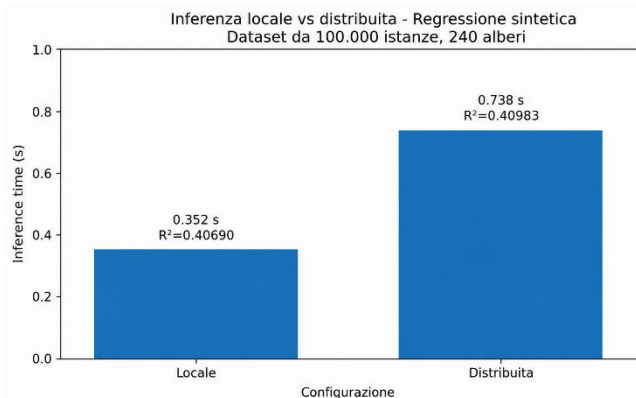
La distribuzione del training di classificazione sintetica a confronto con l'esecuzione locale riduce sensibilmente il tempo di esecuzione mantenendo una qualità predittiva molto vicina alla baseline locale. La differenza di accuracy, come l'esperimento descritto in precedenza, è limitata e accettabile. Lo speedup osservato non è lineare rispetto al numero di worker, comportamento atteso in un sistema distribuito reale. Il tempo complessivo non dipende soltanto dall'addestramento degli alberi, ma anche da overhead di orchestrazione, comunicazione gRPC, accesso allo storage condiviso, serializzazione degli artifact e sincronizzazione dello stato del job. Inoltre, con alberi relativamente poco profondi, il carico computazionale di ogni singolo task può non essere abbastanza elevato da compensare completamente il costo della distribuzione.

CONFRONTO LOCALE-DISTRIBUITO: INFERENZA

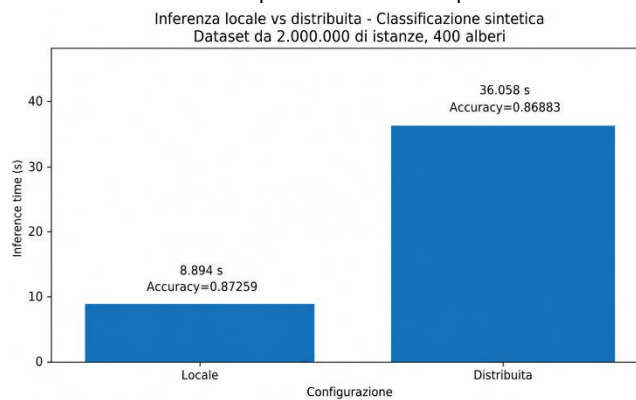


Nel benchmark di inferenza sul dataset sintetico di regressione, eseguito su 400.000 istanze di input, 400 alberi e 8 worker nella configurazione 2x4, l'esecuzione distribuita mostra un comportamento favorevole rispetto alla baseline locale. In particolare, l'inferenza locale richiede 14.48 s, mentre l'inferenza distribuita completa l'elaborazione in 8.42 s, corrispondenti a uno speedup di circa 1.72x. Il modello distribuito mantiene inoltre una qualità predittiva coerente, con $R^2 = 0.42636$, $MAE = 149.84$ e $RMSE = 188.99$.

Questo risultato indica che, su un carico di inferenza sufficientemente ampio, la distribuzione del lavoro tra più worker riesce ad ammortizzare gli overhead di coordinamento e comunicazione.



Abbiamo notato una differenza rilevante rispetto al caso più piccolo sul dataset sintetico di regressione da 100.000 istanze, mostrato nel grafico superiore. In questo scenario, i risultati indicano che l'inferenza distribuita non è conveniente, poiché il costo di coordinamento supera il beneficio del parallelismo.



Nel grafico di confronto di inferenza per classificazione binaria usando il dataset sintetico si può notare che l'inferenza locale completa l'elaborazione in 8.89 s, mentre l'inferenza distribuita richiede 36.06 s. Il modello distribuito mantiene comunque una buona qualità predittiva, con accuracy = 0.86883.

In questo caso, l'inferenza distribuita non risulta efficiente, ma è assolutamente da tenere conto che l'esecuzione dell'allenamento e conseguente inferenza sono stati eseguiti per questo task con modalità 2x4, ed abbiamo riscontrato che questa configurazione, con più di 2 container per istanza, risulta molto lenta.

VALUTAZIONE DELLA FAULT TOLERANCE

I. WORKER

Oltre alle misurazioni di scalabilità, il sistema è stato valutato rispetto alla capacità di tollerare guasti durante l'esecuzione dei job distribuiti. L'obiettivo di questa valutazione è verificare che un'interruzione parziale dell'architettura non comporti necessariamente la perdita completa del lavoro svolto e non richieda il riavvio manuale dell'intero sistema. A differenza di un semplice test basato sull'interruzione generica di un container, il progetto introduce un meccanismo di fault injection che consente di simulare guasti controllati in specifici punti critici del flusso applicativo.

Il supporto alla sperimentazione della fault tolerance è implementato tramite un modulo di chaos testing, configurabile attraverso variabili d'ambiente. Il meccanismo permette di specificare un punto di iniezione del guasto, un'azione da eseguire e un eventuale ritardo prima dell'attivazione.

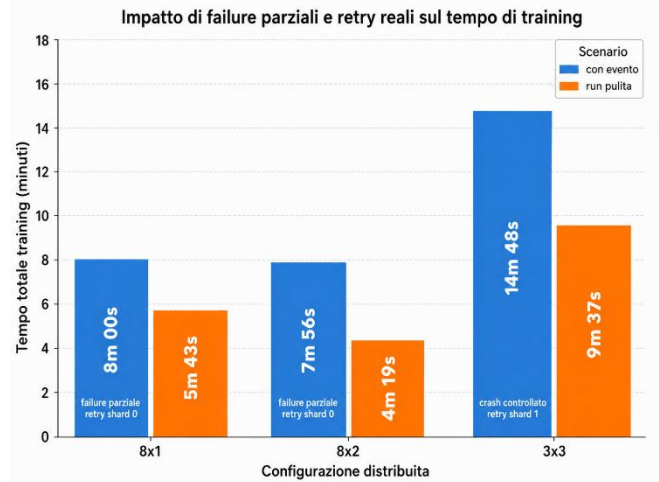
Le azioni supportate includono:

- perdita di heartbeat del worker
- crash prima dell'inizio del training
- crash dopo l'addestramento ma prima della persistenza dell'artifact
- crash dopo la registrazione del progresso
- fallimento durante la predizione distribuita

Le azioni sono state implementate nei seguenti modi:

1. *Fallimento prima dell'invio dell'heartbeat.* In questo caso il worker smette di comunicare regolarmente con il master leader, che può quindi rilevare il nodo come non più affidabile o non più disponibile. Questo scenario verifica la capacità del sistema di monitorare lo stato dei worker e di escludere dalla pianificazione i nodi che non forniscono più segnali di attività.
2. *Crash del worker dopo l'assegnazione dello shard ma prima dell'avvio effettivo del training.* In questa condizione il master può aver già assegnato un sottoinsieme di alberi a un worker, ma nessun artifact è ancora stato prodotto. Il sistema deve quindi essere in grado di riconoscere che il task assegnato non ha generato progresso durevole e deve ripianificare gli alberi mancanti su altri worker disponibili.
3. *Fallimento dopo l'addestramento di un albero ma prima della scrittura dell'artifact nello storage condiviso.* In questo caso il lavoro computazionale è stato svolto localmente dal worker, ma non è stato reso durevole. Dal punto di vista del sistema distribuito, l'albero non può quindi essere considerato completato. Il test verifica che il master non assuma come valido un risultato non persistito e che gli alberi mancanti vengano nuovamente pianificati. Questo comportamento evidenzia il ruolo dello storage condiviso come confine di durabilità: solo gli artifact effettivamente scritti e registrati possono essere usati per ricostruire lo stato del job.
4. *Fallimento dopo la scrittura dell'artifact e dopo l'aggiornamento del progresso del task.* In questa situazione una parte del lavoro può essere già stata completata e resa disponibile sullo storage condiviso. Il sistema deve quindi evitare di perdere il progresso già acquisito e deve distinguere tra alberi effettivamente mancanti e alberi già prodotti. Questo test valuta l'idempotenza del processo di salvataggio degli artifact e la capacità del recovery planner di riprendere l'esecuzione senza ricalcolare inutilmente il lavoro già completato.
5. *La fault tolerance è stata valutata anche nella fase di validazione, testing e inferenza distribuita.* In questo caso il guasto

può essere simulato prima dell'esecuzione del predittore lato worker. Il master assegna a ciascun worker un sottoinsieme degli alberi da valutare e raccoglie predizioni parziali. Se un worker fallisce durante questa fase, il sistema deve rilevare il fallimento della chiamata remota e tentare di affidare lo shard di predizione a un altro worker disponibile. Questo scenario è rilevante perché dimostra che la tolleranza ai guasti non riguarda soltanto il training, ma anche le fasi successive di utilizzo del modello.



II. MASTER

Crash del master leader durante un job di training. In questo scenario viene interrotto il nodo master che sta coordinando l'esecuzione del job. Poiché il sistema adotta un cluster composto da tre nodi master, la perdita del leader non comporta necessariamente l'interruzione definitiva dell'elaborazione. I master rimanenti partecipano a un meccanismo di leader election basato su Raft e, se viene raggiunta la maggioranza, eleggono automaticamente un nuovo leader.

Raft è un algoritmo di consenso distribuito progettato per consentire a un insieme di nodi di concordare su decisioni comuni anche in presenza di guasti parziali. Nel caso generale, Raft può essere utilizzato sia per l'elezione del leader sia per la replicazione consistente di una macchina a stati. Nella nostra implementazione, tuttavia, Raft viene impiegato principalmente per la leader election: esso determina quale master debba assumere il ruolo di coordinatore attivo, mentre la ricostruzione dello stato applicativo non avviene tramite la replica completa dello stato interno dei master, ma attraverso la lettura dello stato persistito su storage condiviso.

Dal punto di vista della tolleranza ai guasti, un cluster Raft composto da (N) nodi richiede una maggioranza pari a:

$$[quorum = \left\lfloor \frac{N}{2} \right\rfloor + 1]$$

Nel caso specifico del sistema realizzato, con (N = 3) master, il quorum necessario è pari a:

$$[quorum = \left\lfloor \frac{3}{2} \right\rfloor + 1 = 2]$$

Di conseguenza, il cluster può tollerare il crash di un nodo master, poiché i due nodi superstiti sono ancora sufficienti per formare una maggioranza ed eleggere un nuovo leader. In termini generali, il numero massimo di guasti tollerabili mantenendo il quorum è:

$$[f = \left\lfloor \frac{N-1}{2} \right\rfloor]$$

Pertanto, con tre master, il sistema tollera:

$$[f = \left\lfloor \frac{3-1}{2} \right\rfloor]$$

guasto master. Se invece due master risultassero contemporaneamente indisponibili, il nodo rimanente non sarebbe più in grado di raggiungere la maggioranza e non potrebbe procedere autonomamente all'elezione di un leader valido.

A seguito dell'elezione, il nuovo leader assume il controllo del sistema e avvia la procedura di recovery leggendo lo stato persistito su Amazon EFS. La recovery non si basa quindi sulla replica completa dello stato applicativo tramite Raft, ma sulla combinazione tra leader election, persistenza su storage condiviso e ricostruzione dello stato a partire da job repository, task ledger e artifact durevoli.

La persistenza su Amazon EFS svolge un ruolo centrale nella fault tolerance del sistema. Dataset preparati, split di training, validation e test, alberi addestrati, metriche, metadata, ledger dei task e manifest dei modelli non rimangono vincolati alla memoria volatile del master o del worker che li ha generati, ma vengono salvati in uno spazio condiviso accessibile dai nodi superstiti.

Dopo un crash, il nuovo leader può quindi ispezionare lo stato persistito, individuare gli alberi già completati, distinguere i task conclusi da quelli mancanti o interrotti e pianificare nuovamente soltanto il lavoro necessario.

Questo approccio riduce la perdita di progresso computazionale e rende il sistema più robusto rispetto a una soluzione con singolo master. Il ruolo di Raft è garantire che, in presenza di un guasto del leader, esista un solo nuovo coordinatore legittimo; il ruolo di EFS è invece garantire che lo stato necessario alla ripresa del job sia accessibile anche dopo la perdita del nodo che stava coordinando l'esecuzione.

L'organizzazione del sistema dimostra che suddividere l'esecuzione in più meccanismi permette al sistema di tollerare guasti parziali, ridurre la perdita di progresso computazionale e proseguire l'esecuzione senza dover ripetere necessariamente l'intero job dall'inizio. Rispetto a una soluzione centralizzata eseguita su un singolo nodo, l'architettura proposta offre quindi una maggiore robustezza operativa e una migliore capacità di recupero in presenza di crash o indisponibilità temporanee dei componenti.

Tabella riassuntiva dei casi di fault tolerance

Scenario	Punto di guasto	Comportamento atteso
Worker heartbeat failure	Prima dell'invio dell'heartbeat	Worker rilevato come stale/non affidabile
Worker crash before trainer	Dopo assegnazione shard, prima training	Shard ripianificato
Worker crash before artifact write	Dopo fit, prima salvataggio albero	Albero considerato mancante e rieseguito
Worker crash after progress update	Dopo salvataggio/progresso	Progresso persistito conservato
Prediction worker failure	Prima del predittore	Shard di predizione riassegnato
Master leader crash	Durante training	Nuova elezione leader e recovery da EFS

CONCLUSIONI E LIMITAZIONI

Il sistema presenta la progettazione e l'implementazione di una Random Forest distribuita basata su architettura Master-Worker, eseguito in ambiente cloud.

I risultati sperimentali mostrano che la distribuzione del training consente di ridurre sensibilmente i tempi di addestramento su dataset di grandi dimensioni, mantenendo una qualità predittiva confrontabile con quella della baseline locale. In particolare, nei confronti su dataset sintetici da 2 milioni di istanze, il sistema distribuito ha prodotto modelli con metriche molto vicine alla controparte locale, ottenendo al tempo stesso una riduzione significativa del tempo di training. Questo conferma che la Random Forest si presta in modo naturale alla distribuzione,

poiché i singoli alberi possono essere addestrati in modo indipendente e successivamente aggregati.

L'analisi delle metriche ML ha inoltre evidenziato l'importanza di una valutazione critica dei dataset. Lo scenario originale del dataset sanitario produce prestazioni artificialmente elevate a causa della presenza di feature potenzialmente soggette a data leakage.

Gli scenari no-leakage, noise injection, class imbalance e classificazione multiclasse hanno invece permesso di valutare il comportamento del modello in condizioni più realistiche e complesse. In particolare, gli esperimenti hanno mostrato che l'accuracy da sola non è sempre sufficiente, soprattutto in presenza di classi sbilanciate o minoritarie; metriche come balanced accuracy, macro F1-score e recall per classe risultano fondamentali per interpretare correttamente le prestazioni del modello.

Dal punto di vista distribuito, il sistema ha dimostrato una buona robustezza rispetto a guasti parziali. I test di fault tolerance e chaos injection hanno verificato il comportamento del sistema in presenza di crash dei worker, perdita di heartbeat, fallimenti in punti critici del training e crash del master leader. La combinazione tra suddivisione del lavoro in shard indipendenti, task ledger persistente, salvataggio degli artifact su EFS e leader election consente al sistema di ridurre la perdita di progresso computazionale e riprendere l'esecuzione senza dover necessariamente ricominciare l'intero job.

Sono emersi tuttavia anche alcuni limiti. Inoltre, l'inferenza distribuita non risulta sempre più conveniente dell'inferenza locale: su batch piccoli o medi, il costo della comunicazione e del coordinamento può superare il beneficio del parallelismo. Questo risultato chiarisce che la distribuzione è particolarmente vantaggiosa per il training su dataset grandi, mentre per l'inferenza deve essere valutata in funzione della dimensione del batch e del carico computazionale effettivo.

SVILUPPI FUTURI

Per quanto riguarda gli sviluppi futuri, una prima estensione riguarderebbe l'introduzione di un meccanismo di bootstrapping gestito lato master, finalizzato al calcolo dell'Out-Of-Bag error. Attualmente il sistema distribuisce ai worker gli shard di alberi da addestrare e demanda a essi la costruzione dei singoli stimatori. Una possibile evoluzione consiste nel far generare al master, in modo controllato e riproducibile, i campioni bootstrap associati ai diversi alberi, mantenendo traccia anche delle istanze escluse da ciascun campione. Tali istanze, definite out-of-bag, potrebbero essere utilizzate per stimare l'errore del modello senza richiedere necessariamente un validation set separato. Questo approccio risulterebbe particolarmente utile su dataset piccoli, nei quali sottrarre una quota significativa di dati per validation o test può ridurre la quantità di esempi disponibili per il training.

Una seconda direzione di sviluppo riguarda l'integrazione di un sistema di autotuning degli iperparametri. Il master potrebbe generare automaticamente un insieme di configurazioni candidate, variando alcuni parametri rilevanti della Random Forest, come numero di alberi, profondità massima, numero massimo di feature considerate a ogni split, criterio di split e soglie minime di campioni per nodo o foglia. Ogni configurazione potrebbe essere trattata come un esperimento indipendente, distribuito sui worker e valutato tramite una metrica obiettivo definita in base al task, ad esempio accuracy, macro F1-score, balanced accuracy o R^2 . Al termine della valutazione, il sistema selezionerebbe automaticamente il modello migliore e pubblicherebbe il relativo manifest come modello finale. In questo modo, l'architettura non si limiterebbe a distribuire il training di una singola configurazione, ma diventerebbe una piattaforma per l'esplorazione automatica dello spazio degli iperparametri.

Un ulteriore sviluppo possibile riguarda l'aumento della ridondanza del cluster master. L'architettura attuale prevede più nodi master con leader election, ma potrebbe essere estesa

avviando più container master distribuiti sulle diverse istanze EC2 disponibili. Questo permetterebbe di incrementare il numero di repliche del control plane e di valutare il comportamento del sistema in scenari più estesi e con maggior rischio di guasti lato control plane. Una configurazione di questo tipo consentirebbe di studiare meglio l'impatto del numero di master sulla disponibilità del sistema, sui tempi di elezione del leader e sulla capacità di mantenere continuità operativa in presenza di guasti multipli o riavvii di singoli container.

Un possibile sviluppo futuro riguarda la gestione dei worker rilevati come zombie. Nell'implementazione attuale, quando il master identifica un worker in uno stato non affidabile, la politica adottata consiste nel metterlo in quarantena per l'allenamento corrente, evitando che riceva nuovi shard nello stesso job. Questa soluzione consente di proteggere l'esecuzione in corso, ma potrebbe essere estesa con una politica di quarantena più strutturata e persistente.

Una possibile evoluzione consiste nell'introdurre un sistema di reputation score associato a ciascun worker, mantenuto dal master anche oltre il singolo job. Il punteggio verrebbe aggiornato dinamicamente in base al comportamento del nodo, penalizzando timeout, crash e stati zombie, e premiando invece il completamento corretto dei task. In questo modo, il sistema potrebbe privilegiare i worker più affidabili nella pianificazione e limitare automaticamente l'utilizzo di quelli instabili, riducendo il rischio di assegnare nuovi shard a nodi che potrebbero causare ulteriori fallimenti o rallentamenti.

REFERENCES

- [1] Amazon Web Services, AWS Documentation, <https://docs.aws.amazon.com/>
- [2] Trees and Forests, Machine Learning, Gabriele Russo Russo [6_trees.pdf](#)
- [3] Consensus in Distributed Systems, Sistemi Distribuiti e Cloud Computing, Valeria Cardellini [DS_Consensus.pdf](#)